

GEOPROCESSAMENTO • POSTGIS • MAPBIOMAS

Sistema Sossego × Desmatamento

Monitoramento do desmatamento no entorno da **Mina do Sossego**
(Canaã dos Carajás — Pará) com **PostGIS**, **API Python** e **Dashboard Web**

BUFFER 1 km

312,14 ha

impacto direto • cava/pilha

BUFFER 5 km

7.803,61 ha

entorno funcional • defesa
principal

BUFFER 10 km

31.214,45 ha

controle regional • fundo
agropecuário

◆ Frase para banca — memorize

"1 km mede a mina, 10 km mede a região, 5 km mede o entorno de verdade — por isso defendemos 5 km como entorno funcional e 1 km como impacto direto." • O sistema prova com números que o impacto direto da mina é confinado e o entorno é dominado pela pastagem.

Mina	Mina do Sossego (cobre/ouro) — Vale S.A. • Canaã dos Carajás-PA • lon -50,0790 • lat -6,4325 • operação desde 2004
Fonte	MapBiomas Coleção 10 • resolução 30 m • anos 2008 • 2015 • 2023 • classes mineração (30) • pastagem (15) • outros • PRODES/INPE como validação
Stack	PostgreSQL 18 + PostGIS 3.6 • Python 3.14 + psycopg 3 + http.server (sem framework) • Leaflet 1.9 + Chart.js 4 • Bootstrap 5 • QGIS + shp2pgsql • pgAdmin4
Acesso	http://localhost:8000/login.html • VM NAT 15432→5432 • logins admin/123 (professor) • aluno/123 (aluno) • token Bearer

Documentação técnica completa • setembro 2026 • Projeto Sossego • Disciplina: Geoprocessamento

Base atual sintética para desenvolvimento — substituir pelo MapBiomas real via **ROTEIRO_QGIS.md** na entrega final. Divergência QGIS × banco aceitável até 2%.

Sumário

01	Visão geral	— objetivo e público	07	Consultas PostGIS	— série + conferência QGIS
02	Arquitetura	— 4 camadas + diagrama	08	Instalação e execução	— VM, carga, servidor e pgAdmin
03	Requisitos	— RF1–RF6 + RNF1–RNF5	09	Testes	— 19 verificações (18/18 + T16b)
04	Modelo de dados	— tabelas, view, índices e áreas	10	Segurança e validação	— sha256, token, SQL parametrizado
05	API REST	— 8 rotas + autenticação Bearer	11	Limitações e futuros	— 30 m, 5000 limite, PRODES, CSV
06	Frontend	— mapa, KPIs, gráfico e interpretação	12	Estrutura de arquivos	— árvore do projeto

■ Como usar esta documentação

• **Apresentação (5 min):** leia §1 + §6 + §4 (frase dos raios). • **Banca técnica:** foque em §7 + §5 + §9. • **Instalação:** siga §8 passo-a-passo. • **Dados reais:** execute **ROTEIRO_QGIS.md** (§8.4) e valide em §7.

Legenda visual usada no documento: ■ Mineração ● Pastagem ● Outros — Buffer 1 km — Buffer 5 km - - Buffer 10 km

1 Visão geral

O sistema cruza a localização da **Mina do Sossego** (cobre/ouro, Vale S.A.) com **manchas de desmatamento vetorizadas do MapBiomias** (anos **2008, 2015 e 2023**) dentro de **três buffers concêntricos (1, 5 e 10 km)**. O objetivo é **quantificar a perda florestal por raio, ano e classe** (mineração, pastagem, outros) e sustentar a interpretação de que **o impacto direto da mina é confinado, enquanto o entorno é dominado pela agropecuária**.

Pergunta central

"O quanto a floresta recuou perto da Mina do Sossego e esse recuo tem a ver com a mineração?"

Público

Alunos e professor da disciplina
perfis aluno e professor no login
admin/123 • aluno/123

Local + SRID

Canaã dos Carajás-PA
lon **-50,0790** • lat **-6,4325**
SRID **31982** (SIRGAS 2000 / UTM 22S)

Por que três raios? A escolha evita o "chute" de um único entorno e cria uma defesa comparativa:

1 km — impacto direto

Cobre cava, pilha de estéril e usina.
É onde a **mineração** aparece de verdade.
312,14 ha

5 km — entorno funcional ★

Vila, acessos, terceiros e pastagens de apoio. Mistura direto + indireto.
7.803,61 ha — resposta principal

10 km — controle regional

Fundo agropecuário (pecuária leiteira antes do cobre). Se desmate explode aqui e mina não, vetor não é a mina.
31.214,45 ha

Períodos comparados: **2008** pré-expansão (mina nova, linha de base) • **2015** operação plena (mina consolidada + boom urbano de Canaã) • **2023** retrato atual. Três pontos mostram **tendência**, não só antes/depois. **MapBiomias** foi escolhido sobre PRODES por ter série anual contínua e separar classe mineração (30) de pastagem (15).

2 Arquitetura

Arquitetura em **4 camadas**, 100% reproduzível na VM Ubuntu Server. Sem frameworks — exigência da disciplina (RNF1). Todas as geometrias em **31982** (metros) com saída em **4326** para o Leaflet.

1	Banco PostgreSQL 18 + PostGIS 3.6 VM Ubuntu Server • porta 5432 NAT host 15432 • DB sossego_desmate SRID 31982 • GiST + btree	QGIS extraí dado real MapBiomias shp2pgsql → desmate	pgAdmin4 inspeciona banco host • localhost:15432 ubuntu/123
2	Backend Python puro http.server + psycopg Sem Flask/Django • porta 8000 Token Bearer • SQL parametrizado Validação total de parâmetros	← psycopg →	API :8000 /api/mina • /buffers /desmate • /serie • /resumo

3	Frontend estático Leaflet + Chart.js + Bootstrap 5 login.html • index.html • app.js estilo.css • tema escuro • 75vh	Fetch Bearer GeoJSON 4326 fitBounds • KPIs • Chart	Dashboard Mapa rua/satélite filtros • gráfico • tabela interpretação crítica
--------------	---	---	--

Fluxo de dado real: **QGIS / MapBiomias** —shp2pgsql→ **[PostGIS VM :5432]** ←15432— **pgAdmin4 (host)** ↔ ←psycopg— **API :8000** —**Leaflet**→ **Dashboard (web/)**

Decisões arquiteturais defendidas:

- SRID 31982 (UTM 22S) porque Canaã está na longitude -50 — área e distância saem direto em metros.
- http.server sem framework para provar domínio de SQL e controle manual de rotas/autenticação.
- Leaflet com duas bases (OSM rua + Esri World Imagery satélite) para leitura tanto cartográfica quanto visual da cava.

3 Requisitos

Requisitos funcionais (RF) e não-funcionais (RNF) — mapeados diretamente para rotas, tabelas e testes.

ID	Requisito funcional	Onde se prova
RF1	Cadastrar/autenticar usuários login, senha_hash (sha256), perfil aluno/professor	POST /api/login • GET /api/perfil • tabela usuarios • T1/T2 • login.html
RF2	Expor mina, buffers e manchas em GeoJSON FeatureCollection 4326 para Leaflet	GET /api/mina • /buffers • /desmate • T3-T6 • mapa Leaflet
RF3	Filtrar desmate por ano, classe, buffer e limite validação 400 se inválido	GET /api/desmate?ano&classe;&buffer;&limite; • T7-T11 • filtros index.html
RF4	Série temporal e resumo agregado ha por raio × ano × classe	GET /api/serie (vw_serie_buffer) • /resumo • T12-T14 • gráfico Chart.js
RF5	Cadastrar mancha manual (teste) só professor • valida coord. Brasil	POST /api/desmate {ano, classe, lon, lat, raio_m} • T16-T17 • 403 aluno
RF6	Dashboard com mapa rua/satélite, KPIs, gráfico e interpretação 75vh • OpenStreetMap + Esri • texto crítico	index.html • app.js • interpreta() • KPIs • legenda cores

ID	Requisito não-funcional	Como é atendido
RNF1	Python padrão sem framework	http.server + psycopg puros — prova pede isso • servidor.py 254 linhas
RNF2	Geometrias em 31982 (metros) com saída em 4326	ST_Transform(geom,4326) em todas as rotas GeoJSON • ST_Area/ ST_Buffer em metros
RNF3	Validação de todos os parâmetros	limite 1-5000, classe ∈ {mineracao,pastagem,outros}, ano 1985-2030, buffer 1/5/10, lon -75..-30 lat -35..6
RNF4	Consultas espaciais com índice GiST	mina_geom_idx, buffer_geom_idx, desmate_geom_idx • EXPLAIN mostra Index Scan • 4.62 ms
RNF5	Credenciais nunca em URL (token Bearer)	Authorization: Bearer TOKEN • 401 sem token • T0 • token 32 hex aleatório

Matriz de rastreabilidade: cada RF/RNF tem pelo menos 1 rota + 1 teste + 1 elemento de UI. Ver §9 (testes) para IDs T0-T17 correspondentes.

4 Modelo de dados

Banco **sossego_desmate** — SRID **31982 (SIRGAS 2000 / UTM 22S)**. Todas as geometrias em metros; conversão para 4326 só na saída do Leaflet via **ST_Transform**. Modelo otimizado para **ST_Intersection + ST_Area** por buffer.

Tabela	Descrição	Colunas-chave / Tipo geom	Qtd.
mina	Ponto da cava (lon/lat WGS84 + geom 31982) 1 linha: Sossego • Vale • Canaã dos Carajás	id • nome • operadora • municipio • substancia lon/lat (double) • geom Point 31982	1
buffer_entorno	3 polígonos concêntricos gerados com ST_Buffer(geom, raio_m)	id • mina_id FK • raio_km 1/5/10 UNIQUE geom Polygon 31982	3
desmate	Uma linha por mancha desmatada classe simplificada MapBiomias	id • ano 1985-2030 • classe {mineracao,pastagem,outros} fonte • area_ha numeric(12,2) • geom MultiPolygon 31982	~290*
usuarios	Login do frontend senha_hash sha256 • nunca texto puro	id • login UNIQUE • senha_hash perfil {aluno, professor}	2

Tabela	Descrição	Colunas-chave / Tipo geom	Qtd.
vw_serie_buffer (view)	Agrega ha por raio × ano × classe via ST_Intersection • base do gráfico	raio_km • ano • classe • n_patches ha = SUM(ST_Area(ST_Intersection))/10000	9×3

* 290 patches sintéticos no gerar_base.py (seed 0.42 reproduzível); com dado real o número vem do QGIS. View agrega 3 raios × 3 anos × 3 classes = até 27 linhas.

Índices

GiST em todas as geometrias:
**mina_geom_idx •
buffer_geom_idx
desmate_geom_idx**
+ btree em **desmate(ano)** e
desmate(classe)

EXPLAIN ANALYZE: 4.62 ms
Nested Loop + Index Scan geom &&
buffer

Áreas oficiais dos buffers (ST_Area/10000)

1 km = 312,14 ha | 5 km = 7.803,61 ha | 10 km = 31.214,45 ha
 $\pi \cdot r^2 \cdot 1 \text{ km} = \pi \cdot 1^2 \approx 3,14 \text{ km}^2 = 314 \text{ ha}$ (314,16 teórico; 312,14 com discretização ST_Buffer segmentos=32)

Fórmula no banco: **ROUND((ST_Area(geom)/10000.0)::numeric,2)**

View vw_serie_buffer — coração do dashboard (alimenta gráfico + tabela 1/5/10 km):

```
CREATE OR REPLACE VIEW vw_serie_buffer AS
SELECT b.raio_km, d.ano, d.classe,
COUNT(*) AS n_patches,
ROUND((SUM(ST_Area(ST_Intersection(b.geom, d.geom)))) / 10000.0)::numeric,2) AS ha
FROM buffer_entorno b
JOIN desmate d ON ST_Intersects(b.geom, d.geom)
GROUP BY b.raio_km, d.ano, d.classe;
```

Detalhe: **ST_Intersection** recorta só a porção da mancha dentro do buffer — evita superestimar. Divisão por 10.000 converte m² → ha. Índice GiST em **geom &&** acelera o JOIN espacial.

5 API REST

Base <http://localhost:8000>. Todas as rotas exceto **POST /api/login** exigem cabeçalho **Authorization: Bearer TOKEN** (401 sem token — T0). Erros usam JSON {erro: "..."}.

Sem framework: roteamento manual em **servidor.py** (http.server).

Método & Rota	Parâmetros / Corpo	Resposta	Teste
POST /api/login	{login, senha} JSON • sha256 no banco	200 {token, login, perfil} 400 faltando campo • 401 credencial inválida	T1, T2
GET /api/perfil	Bearer obrigatório —	200 {login, perfil}	—
GET /api/mina	Bearer —	200 FeatureCollection (1 ponto, 4326) ST_Transform(geom,4326)	T3-T5
GET /api/buffers	Bearer —	200 FeatureCollection (3 buffers + area_ha) ORDER BY raio_km • ST_Area/10000	T6
GET /api/desmate	ano, classe, buffer (1/5/10), limite 1-5000 ?ano=2023&classe;=pastagem&buffer;=5&limite;=500	200 FeatureCollection 400 se classe/limite/ano/buffer inválido	T7-T11
GET /api/serie	Bearer —	200 lista {raio_km, ano, classe, n, ha} FROM vw_serie_buffer ORDER BY 1,2,3	T12-T13
GET /api/resumo	Bearer —	200 {mina, por_ano, por_classe, buffers_ha} totais COUNT/SUM + área buffers	T14
POST /api/desmate	{ano, classe, lon, lat, raio_m} só perfil professor • -75 < lon < -30 • -35<lat<6	201 {id} 400 inválido • 403 aluno • 401 sem token	T16-T17 T16b
* /api/*	qualquer método inexistente	404 {erro: "Nao encontrado"} 405 GET /api/login → "Use POST"	T15

■ Autenticação

Fluxo: login.html → POST /api/login {login, senha} → servidor verifica **sha256(senha)** em usuarios → gera **secrets.token_hex(16)** (32 hex) → armazena em TOKENS[token] = {login, perfil} → frontend salva em **localStorage.token** → todas as demais requisições enviam **Authorization: Bearer TOKEN**. Sem token → **401** (T0). Sem URL com senha (RNF5). Logins padrão: **admin/123** (professor, pode POST) e **aluno/123** (aluno, só leitura).

Exemplo de uso (curl):

```
# login
curl -X POST http://localhost:8000/api/login \
  -H "Content-Type: application/json" \
  -d '{"login":"admin","senha":"123"}' # → {token, login, perfil}

# consultar desmate filtrado
curl http://localhost:8000/api/desmate?ano=2023&buffer;=5&limite;=500 \
  -H "Authorization: Bearer SEU_TOKEN"

# cadastrar mancha (só professor)
curl -X POST http://localhost:8000/api/desmate \
  -H "Authorization: Bearer SEU_TOKEN" -H "Content-Type: application/json" \
  -d '{"ano":2023,"classe":"pastagem","lon":-50.05,"lat":-6.45,"raio_m":150}'
```

6 Frontend

Páginas estáticas servidas pelo próprio backend (**http.server** diretório **web/**). Tema escuro, mapa herói 75vh, KPIs claros. Sem build — HTML/CSS/JS puros + CDN.

Página	Conteúdo	Detalhe técnico
login.html	Formulário de acesso; salva token e usuario no localStorage; redireciona para index.html. Se token já existe, vai direto para index.html — use Sair ou aba anônima para ver login	Fetch POST /api/login localStorage.clear() no Sair Enter no campo senha
index.html	Filtros (ano, tipo, distância) • KPIs (área, % do entorno, tipo comum) • mapa • gráfico • tabela 1/5/10 km • interpretação Tema escuro • filtros recarregam na hora (limite 2000)	Bootstrap 5.3 Leaflet 1.9 + Chart.js 4 app.js • estilo.css
app.js	Fetch com Bearer , camadas mina/buffers/desmate, fitBounds, Chart.js, cálculo de % e texto interpretativo. Centróide para pastagem/outros • polígono sólido para mineração	auth() header centroidLatLng() raioCirculo(ha) • interpreta()
estilo.css	Altura do mapa (75vh), controles Leaflet escuros, KPIs claros, fundo degradê no login. #mapa {min-height:75vh; border-radius:.5rem}	.login-fundo gradient .leaflet-bar dark .card-valor 1.5rem

Mapa herói — camadas e estilo visual:

Camada	Visual	Cor / Estilo	Comportamento
Mina	Marcador Leaflet marker	— padrão	Popup: nome • município • substância
Mineração	Área sólida (polígono)	■ #d62728 fill 0.78 • borda #ff3b30 • 1 polígono/ano recortada 1 km	100% dentro 1 km • label "Mina ano · ha"
Pastagem	Círculo no centróide	● #ff9f1c raio 5-16 px proporcional √ha • fill 0.42	Dispersas ao longo de vicinais • popup: classe·ano·ha·fonte
Outros	Círculo no centróide	● #6c757d mesmo raio • menor destaque	Idem pastagem, cor cinza
Buffers	Polígonos concêntricos	— verde 1 km • — azul 5 km • - - cinza 10 km • fill 0.05	Popup: "Buffer X km · ha" • clique mostra área
Base	Rua + Satélite	OSM rua • Esri World Imagery satélite + rótulos • satélite é padrão	Controle canto superior direito • troca instantânea

KPIs (abaixo do mapa): Área desmatada = Σ área_ha do filtro • % do entorno = ha / área_buffer ×100 (ex: 743 / 7803 = 9,53% no sintético) • **Tipo mais comum** = classe com mais manchas no filtro. **Gráfico + tabela:** 3 linhas (1/5/10 km) nos anos 2008/2015/2023 — linha 5 km sobe forte mas 1 km fica baixo → prova expansão do entorno, não da cava.

■ Dica de demonstração na banca

FILTRO 1 (prova visual principal): 2023 + **pastagem** + **5 km** → dezenas de bolinhas laranjas dispersas ao longo de vicinais = vetor agropecuário.
FILTRO 2 (prova confinamento): 2023 + **mineração** + **1 km** → só área vermelha sólida central = mina confinada.
Deixe <http://localhost:8000/login.html> aberto (admin/123) + pgAdmin se perguntarem do banco. Não precisa abrir QGIS na hora — só citar ROTEIRO_QGIS.md.

7 Consultas principais (PostGIS)

Duas consultas sustentam o dashboard e a validação QGIS × banco. Ambas usam **índice GiST** — EXPLAIN mostra Index Scan (4.62 ms).

7.1 Série por buffer — alimenta o gráfico (vw_serie_buffer)

```
SELECT b.raio_km, d.ano, d.classe,
       COUNT(*) AS n,
       ROUND((SUM(ST_Area(ST_Intersection(b.geom, d.geom))) / 10000)::numeric,2) AS ha
FROM   buffer_entorno b
JOIN   desmate d ON ST_Intersects(b.geom, d.geom)
GROUP BY 1,2,3
ORDER BY 1,2,3; -- 1 km → 5 km → 10 km | 2008 → 2015 → 2023 | mineracao → pastagem → outros
```

Leia assim: para cada buffer (1/5/10) cruza com cada mancha que **intersecta** (&& + ST_Intersects com GiST), recorta a interseção (ST_Intersection), soma áreas em m², divide por 10.000 → ha. COUNT = nº de patches. View materializa essa lógica para a API /api/serie.

7.2 Conferência QGIS × banco — divergência aceitável até 2%

```
-- No pgAdmin (após importar shape real):
SELECT raio_km, ano, SUM(ha) AS total_ha
FROM   vw_serie_buffer
GROUP BY 1,2
ORDER BY 1,2;

-- No QGIS (antes de importar): estatísticas por buffer (Field Calculator: $area/10000)
-- Compare: se |qgis - banco| / qgis > 2% → checar SRID da importação (deve ser 31982)
-- e geometrias inválidas com ST_IsValid / ST_MakeValid.
```

Exemplo com base sintética (desenvolvimento) — trocar pelos números do QGIS na entrega final:

Ano	1 km (ha)	5 km (ha)	10 km (ha)	Obs.
2008	6,11	223,04	351,73	linha de base • mina nova
2015	17,48	410,63	1.049,02	operação plena • boom Canaã
2023	29,47	743,67 (9,53% de 7.803)	2.532,66	retrato atual • pastagem domina

Totais por classe (sintético): **pastagem 2.957,34 ha** vs **mineração 609,73 ha** vs outros 637,19 ha → pastagem ≈ 5× mineração. Mesmo padrão deve aparecer no dado real.

■ Atenção — números sintéticos

Todos os valores acima são da **base sintética de desenvolvimento** (seed 0.42 em gerar_base.py) para permitir desenvolver sem baixar GBs do MapBiomas. Na entrega final valem os números extraídos no QGIS pelo ROTEIRO_QGIS.md (§8.4). A divergência entre QGIS e banco deve ficar ≤ 2% (borda de interseção).

8 Instalação e execução

Ordem correta: **VM** → **carga** → **servidor** → **testes** → **pgAdmin** → **dado real (opcional)**. Todo o setup cabe em 10 minutos se a VM já tiver PostGIS.

1	VM (1ª vez) — PostGIS já instalado Ubuntu Server • acesso do host via NAT	Dentro da VM: CREATE DATABASE sossego_desmate OWNER ubuntu; \c sossego_desmate CREATE EXTENSION postgis; No VirtualBox → Rede → NAT → Portas: 15432 → 5432 (banco) e 2222 → 22 (ssh)
2	Carga — base sintética gera mina + 3 buffers + patches + usuários	python3 -m venv .venv source .venv/bin/activate pip install 'psycopg[binary]' python teste.py # PostGIS ok? python gerar_base.py # mina + buffers + base DSN tenta: 15432 → localhost → postgres Logins criados: admin/123, aluno/123
3	Servidor e testes no host, com a VM ligada	python servidor.py # → http://localhost:8000/login.html python testes.py # → 18/18 + T16b = 19/19 Servidor: http.server diretório web/ Testes validam auth + geo + filtros

4	pgAdmin4 (host) inspeção visual do banco	Novo servidor: Host localhost • Porta 15432 Usuário ubuntu • Senha 123 Banco sossego_desmate Ver tabelas mina/buffer/desmate Rodar SELECT em vw_serie_buffer
5	Dado real (entrega final) troca sintético → MapBiomias	Seguir ROTEIRO_QGIS.md : 1. Buffer 1/5/10 km em 31982 2. Recorte + poligonização MapBiomias 2008/2015/2023 3. shp2pgsql para tabela desmate shp2pgsql -s 31982 -g geom desmate.shp desmate psql -h localhost -p 15432 ...

Comandos rápidos (copie e cole):

```
# --- VM ligada ---
source .env/bin/activate
python gerar_base.py      # recria tudo (idempotente)
python servidor.py        # http://localhost:8000/login.html (admin/123 • aluno/123)
# outro terminal:
python testes.py          # deve imprimir 19/19 (18 + T16b)

# --- conferência no pgAdmin / psql ---
SELECT raio_km, ano, SUM(ha) FROM vw_serie_buffer GROUP BY 1,2 ORDER BY 1,2;
SELECT * FROM desmate LIMIT 5;
SELECT postgis_version(); -- deve retornar 3.6+
```

9 Testes

testes.py executa **19 verificações** automatizadas (18 originais + T16b aluno 403). Todas devem passar antes da entrega. Cobrem autenticação, GeoJSON, filtros, série, resumo e cadastro.

ID	O que testa	Requisição	Esperado
T0	401 sem token proteção Bearer	GET /api/mina (sem header)	401 {erro:"Nao autenticado"}
T1	Login errado 401	POST /api/login {aluno, senha errada}	401
T2	Login ok + token	POST /api/login {aluno,123}	200 {token, login, perfil}
T3	Mina é FeatureCollection	GET /api/mina Bearer	200 type=FeatureCollection
T4	Mina tem 1 ponto Sossego	GET /api/mina	features.length == 1
T5	Mina lon/lat no Pará	GET /api/mina	-51 < lon < -49 -7.5 < lat < -5.5
T6	3 buffers	GET /api/buffers	features.length == 3 1/5/10 km
T7	Limite respeitado	GET /api/desmate?limite=5	5 features
T8	Filtro ano	GET /api/desmate?ano=2023	todos ano==2023
T9	Filtro buffer	GET /api/desmate?buffer=1	count > 0 só intersecta 1 km
T10	Classe inválida 400	GET /api/desmate?classe=dragao	400 {erro}
T11	Limite inválido 400	GET /api/desmate?limite=abc	400
T12	Série é lista	GET /api/serie	list len ≥ 9
T13	Série tem ha	GET /api/serie	todo item tem "ha"
T14	Resumo completo	GET /api/resumo	por_ano + por_classe + buffers_ha + mina
T15	404 rota inexistente	GET /api/nao-existe	404
T16	POST 201 + id (professor)	POST /api/desmate {2023, pastagem, -50.05,-6.45,150} token admin	201 {id:int}

ID	O que testa	Requisição	Esperado
T16b	Aluno 403	POST /api/desmate token aluno	403 "Apenas professor"
T17	POST incompleto 400	POST /api/desmate {ano:2023} token admin	400

Resultado atual: 19/19 (validado 2026-09-21 via `python testes.py`; 18/18 originais + T16b). Se algum falhar, verifique: VM ligada? porta 15432 redirecionada? token expirado? `gerar_base.py` rodou?

Como rodar:

```
python testes.py
# saída esperada:
# PASSOU T0 sem token devolve 401
# PASSOU T1 login errado 401
# ...
# PASSOU T16 POST 201 + id
# PASSOU T16b aluno POST 403
# PASSOU T17 POST incompleto 400
# -----
# RESULTADO: 19 / 19 testes passaram
```

10 Segurança e validação

Requisitos de segurança da disciplina (autenticação, SQL injection, validação de entrada) — todos cobertos e testados.

Item	Como é feito	Onde / Teste
Senhas	sha256 hex — nunca texto puro <code>hashlib.sha256(senha.encode()).hexdigest()</code> • col. <code>usuarios.senha_hash</code>	<code>gerar_base.py</code> • <code>servidor.py:sha()</code> T1
Token	Aleatório 32 hex por sessão • <code>secrets.token_hex(16)</code> • dict TOKENS em memória	POST /login → {token} Bearer em todas rotas
SQL 100% parametrizado	Nenhum f-string em SQL • <code>c.execute("... WHERE login=%s", (login,))</code> • anti-injection	<code>servidor.py</code> T10/T11 não injetam
Limite	$1 \leq \text{limite} \leq 5000$ • <code>isdigit()</code> + range	GET /desmate?limite T11 → 400
Classe	$\in \{\text{mineracao, pastagem, outros}\}$ • set CLASSES	T10 → 400
Ano	$1985 \leq \text{ano} \leq 2030$ • CHECK <code>desmate.ano</code>	POST /desmate → 400
Buffer	$\in \{1, 5, 10\}$ • <code>int</code> + $\in$	GET /desmate?buffer 400
Coordenadas	$-75 < \text{lon} < -30$ e $-35 < \text{lat} < 6$ (Brasil)	POST /desmate → 400
Métodos	405 GET /api/login → "Use POST" • 404 /api/*	<code>servidor.py</code> T15
Permissão	Só professor pode POST • perfil $\in \{\text{professor, admin}\}$	T16b aluno → 403

■ Credenciais de demonstração (não use em produção)

admin / 123 perfil professor — pode cadastrar mancha (POST /desmate) • aluno / 123 perfil aluno — só leitura (GET). Senhas em sha256 no banco, nunca em URL. Token expira quando servidor reinicia (memória). Para resetar: `python gerar_base.py` recria usuários.

11 Limitações e trabalhos futuros

Transparência sobre o que o trabalho não faz — valorizado na banca.

Limitação atual	Por que existe	Evolução proposta
Base sintética (desenvolvimento)	Evita baixar GBs do MapBiomias para desenvolver frontend/API seed 0.42 reproduzível	Trocar pelo MapBiomias real via QGIS ROTEIRO_QGIS.md + shp2pgsql → tabela <code>desmate</code>

Limitação atual	Por que existe	Evolução proposta
Resolução 30 m não capta clareiras pequenas	Limite do MapBiomias Coleção 10 • pixel 30×30 m clareira <900 m² invisível	Complementar com imagens 10 m (Sentinel-2) ou drone para áreas críticas 1 km
Sem paginação geográfica (limite 5000)	Simplicidade didática evita cursor/paging /desmate?limite=5000 max	Paginação + cluster no Leaflet ou vector tiles (MVT)
3 anos (2008/2015/2023) em vez de série completa	Foco na tendência sem sobrecarregar demo antes/durante/atual	Série 1985-2024 completa + animação temporal no mapa

Trabalhos futuros (roadmap):

- **PRODES/INPE como segunda fonte** — validar incrementos anuais oficiais do INPE contra MapBiomias; bom para auditoria cruzada, mas sem separação fina mineracao×pastagem.
- **Série completa 1985–2024 + animação temporal** — slider no Leaflet mostrando evolução ano a ano; exportar GIF para apresentação.
- **Exportação CSV do resumo** — botão "**Baixar CSV**" no dashboard (por_ano + por_classe + buffer_ha) para análise no Excel/QGIS.
- **Análise de distância à cava (ST_Distance)** — histograma de desmate vs. distância; prova estatística de confinamento.
- **Detecção de forma** — cava = geometria compacta/geométrica vs. pasto = retangular ao longo de vicinais; classificador morfológico.

◆ Mensagem para a banca — como defender as limitações

"Sabemos que a base atual é **sintética** — ela existe para provar que o **sistema funciona** (API + PostGIS + dashboard). O valor científico vem da **troca pelo MapBiomias real no QGIS** (ROTEIRO_QGIS.md), onde a resolução 30 m é a melhor disponível gratuitamente e a divergência de 2% é aceitável. A evolução com **PRODES + série completa + CSV** já está mapeada."

12 Estrutura de arquivos

Árvore do projeto — todos os arquivos essenciais para reproduzir o sistema do zero.

```
projeto/
■■■ schema.sql           # modelo PostGIS 31982 (mina, buffers, desmate, usuarios, view)
■■■ gerar_base.py        # cria mina + buffers 1/5/10 km + desmate sintético + usuarios
■■■ servidor.py          # API http.server + pycopp (porta 8000, Bearer)
■■■ teste.py             # check postgis_version
■■■ testes.py            # 19 verificações (T0-T17 + T16b)
■■■ setup_vm.sh          # roda DENTRO da VM (PostGIS + NAT)
■■■ README.md            # instruções curtas
■■■ ROTEIRO_QGIS.md       # passo-a-passo dado real (buffers + MapBiomias + shp2pgsql)
■■■ ENTREGA.txt          # números finais 1/5/10 km + % + interpretação (para professor)
■■■
■■■ web/
■   ■■■ login.html       # tela de acesso (token → localStorage)
■   ■■■ index.html       # dashboard: filtros + mapa 75vh + KPIs + gráfico + tabela
■   ■■■ app.js           # fetch Bearer, Leaflet, Chart.js, fitBounds, interpreta()
■   ■■■ estilo.css       # mapa, controles escuros, KPIs, login degradê
■■■
■■■ PDF/
■   ■■■ Documentacao_Sistema_Sossego.pdf # ← este arquivo (técnica completa)
■   ■■■ Resumo_Sistema_Sossego.pdf      # resumo 2 págs (o que faz + essencial)
■   ■■■ Guia_Estudo_Sossego.pdf          # guia de estudo 3 págs (defesa na banca)
■   ■■■ Guia_Apresentacao_Front_Sossego.pdf# guia só-front (o que mostrar em 5 min)
■■■
■■■ .venv/                  # venv com pycopp[binary]
```

Camada	Tecnologia	Versão / Detalhe
Banco	PostgreSQL + PostGIS	PostgreSQL 18 + PostGIS 3.6 • VM Ubuntu Server
Backend	Python + pycopp	Python 3.14 • pycopp 3 [binary] • http.server (sem framework)
Frontend mapa	Leaflet	1.9.4 • OpenStreetMap + Esri World Imagery + rótulos
Frontend gráfico	Chart.js	4.4.1 • line chart 3 séries (1/5/10 km)

Camada	Tecnologia	Versão / Detalhe
Estilo	Bootstrap	5.3.0 • tema escuro • cards + badges
GIS desktop	QGIS + shp2pgsql	QGIS (fora da VM) • shp2pgsql -s 31982 -g geom
Admin DB	pgAdmin4	Host • localhost:15432 • ubuntu/123

Reprodução em 3 comandos (após VM pronta): `python gerar_base.py` → `python servidor.py` → `python testes.py` • Dashboard: `http://localhost:8000/login.html` • Logins: **admin/123** (professor) • **aluno/123** (aluno)

◆ Em 1 frase para a banca — feche assim

"O sistema prova com números e mapa que o impacto direto da mina é confinado no 1 km (cava/pilha) e que o desmatamento do entorno — 5 km funcional, 9,53% desmatado, dominado por pastagem 5× maior que mineração — é majoritariamente agropecuário, coerente com a história de Canaã dos Carajás."

Documentação gerada automaticamente • Projeto Sossego • Mina do Sossego — Vale S.A. • Canaã dos Carajás-PA • MapBiomias Coleção 10 • SRID 31982 • sossego_desmate • setembro 2026 • v1.0 — base sintética de desenvolvimento; substituir pelos números QGIS na entrega final via ROTEIRO_QGIS.md

Referências: MapBiomias (mapbiomas.org) • PRODES/INPE (terrabrasilis.dpi.inpe.br) • PostGIS (postgis.net) • Leaflet (leafletjs.com) • Chart.js (chartjs.org) • SIRGAS 2000 / UTM 22S (EPSG:31982)